



# Sharing Research Software

Reproducible, Citable, and Discoverable

Tim Dennis  
UCLA OSPO

Leigh Phan  
UCLA OSPO

Karla Padilla  
UCSD OSPO

Reid Otsuji  
UCSD

Laura Langdon  
UC OSPO

2026-09-23

# Welcome!

---

UC Carpentries Workshop Series · Fall 2026

We'll begin shortly. Grab the collaborative notes and settle in.

[pad.carpentries.org/2026-fall-uc-sharing-research-software](https://pad.carpentries.org/2026-fall-uc-sharing-research-software) · [ucospo.net/research-software-citable-discoverable](https://ucospo.net/research-software-citable-discoverable)

# Code of Conduct

---

## We agree to

- Use **welcoming and inclusive** language
- Be **respectful** of different viewpoints and experiences
- Accept **constructive criticism** gracefully
- Show **courtesy and respect** toward everyone here

## If something's wrong

- Tell an **instructor** or the **host** — chat DM or email, privately
- Or report to the **CoC Committee**:  
**coc@carpentries.org**
- All reports are **kept confidential** and taken seriously

Full text & report form: [docs.carpentries.org/policies/coc](https://docs.carpentries.org/policies/coc)

# How we learn together

---

## Our pedagogy

- **Live coding:** we learn by doing together, and mistakes are teaching moments, not detours
- **Mental models:** new skills land when we connect them to work you already do
- **No overload:** a few key concepts done well beats racing through everything

## Managing expectations

- **Practical competence:** skills you can use in your role right away, not exhaustive coverage
- **Collaborative pace:** we move as a group, and your questions help set the speed
- **Mistakes are expected:** coding is troubleshooting and persistence, for all of us

# Software Is the Missing Scholarly Output

# The question that starts this lesson

---

“The journal accepted my paper, but they want a citable version of the code, with a DOI. How do I do that?”

Ten years ago, the same question arrived about **datasets** — and the library built the answer: a persistent identifier, a catalog record, a citation format, a preservation copy. Software is the third thing modern research runs on, and mostly hasn’t gotten that treatment yet.

**You write research software**, and want to make your own code citable, reusable, and discoverable.

**You support people who do**, as a librarian, OSPO member, or research-support specialist.

Watch for **Supporting others** call-outs — they translate each step into advising and consultation work.

# How common is the software citation gap?

---

Research software is fundamental to modern scholarship, but challenges remain around its discoverability, reproducibility, and sustainability.

**59.5%**

of 3.2 million PubMed Central publications mentioned at least one software item

**38%**

of 2020 publications gave enough detail to **uniquely identify** the software used

Barker et al. (2022) doi:10.1038/s41597-022-01710-x · Schindler et al. (2022) doi:10.7717/peerj-cs.835

# A GitHub URL isn't a citation

---

## The wrong way

“We used the analysis script from `https://github.com/UC-OSP0-Network/software-demo`”

- URLs break (link rot)
- No version specified — which run produced which result?
- No formal credit; the author's name isn't even visible
- Cannot appear in citation metrics

# The right way

---

Phan, L., Padilla, K., Otsuji, R., & Dennis, T. (2026). *Biodiversity Analysis Toolkit* (v0.1.0). Zenodo. <https://doi.org/10.5281/zenodo.123456>

- DOI is permanent — survives username changes, repo moves, even GitHub disappearing
- Version is explicit, so someone can reproduce the exact run
- Author gets formal credit and citation metrics

The fix is a `CITATION.cff` file and a DOI — both of which you'll create in this workshop.

# Software has a durability problem

---

## The Bus Factor

The number of people who need to be “hit by a bus” before a project becomes unmaintainable.  
**For most research code, the answer is one.**

This isn't hypothetical, even for critical infrastructure. Before 2014, OpenSSL secured about a sixth of the web's encrypted traffic and ran on ~**\$2,000/year** in donations, with **one person paid** to maintain it. Then Heartbleed happened.

Core Infrastructure Initiative, Linux Foundation. [en.wikipedia.org/wiki/Core\\_Infrastructure\\_Initiative](https://en.wikipedia.org/wiki/Core_Infrastructure_Initiative)

# The fix: FAIR4RS principles

---

**F — Findable** DOI, CITATION.cff, metadata

**A — Accessible** Public GitHub, archived on Zenodo

**I — Interoperable** Standard formats, documented dependencies

**R — Reusable** LICENSE, README, environment files

# Before → after

---

## Before ( main )

- ❌ No LICENSE
- ❌ No citation information
- ❌ No DOI
- ❌ Minimal README
- ❌ No community or metadata files

## After ( 05-add-metadata )

- ✅ LICENSE (BSD-3-Clause)
- ✅ CITATION.cff
- ✅ Complete README
- ✅ CONTRIBUTING.md, CODE\_OF\_CONDUCT.md
- ✅ .zenodo.json, DOI badge

**This is what makes software Findable, Accessible, Interoperable, and Reusable.**

# Real-world example: Spack

---

Used by national labs, universities, and supercomputing centers: [github.com/spack/spack](https://github.com/spack/spack)

-  README with clear description and getting-started guide
-  LICENSE (MIT and Apache-2.0)
-  CITATION.cff file
-  Documentation
-  Code of conduct and contributing guide

# What we'll build together

---

Starting from the bare `main` state in your fork, progressively:

1. Sharing research software (*exploring the starting repository*)
2. Adding an open-source license
3. Creating a CITATION.cff file
4. Minting a DOI and creating a release
5. Improving metadata and discoverability
6. Wrap-up and reflection

View-only reference branches ( `02-add-license` , `03-add-citation` , `04-create-release` , `05-add-metadata` ) let you check your work at any point.

# Sharing Research Software Effectively

# Present, but not yet usable

---

Every archivist knows the unprocessed accession: boxes that made it into the building but not into the collection — technically held, practically invisible.

Most research software on GitHub is in exactly that state. **Public is not the same as shared.**

An open repository with no license, no citation, no usable description is like leaving the reading room unlocked and calling it access: people can technically get in, and nothing helps them once they do.

# Fork the demo repository

---

Every hands-on step happens in **your own fork**, not the shared demo repo.

1. Open `github.com/UC-OSP0-Network/software-demo`
2. Click **Fork** → **Create fork** (defaults are fine — `main` branch only)
3. Confirm the top-left reads `YOUR-USERNAME/software-demo` , not `UC-OSP0-Network/software-demo`

Reference branches are there to check your work, never to build on. Full setup details: [ucospo.net/research-software-citable-discoverable/learners/setup](https://ucospo.net/research-software-citable-discoverable/learners/setup)

# The starting point

---

```
1 software-demo/  
2 |— README.md  
3 |— requirements.txt  
4 |— src/  
5   |— analysis.py
```

This repository intentionally *lacks* many elements of good research software practice. By the end of the lesson, it will include licensing, citation metadata, improved discoverability, and versioning.

# Supporting others: a four-question triage

---

When a researcher brings you their repo, you're doing a fast triage, not a full code review — a reference interview, not a dissertation defense.

1. **Rights** — is there a license? Can this legally be reused at all?
2. **Credit** — can a stranger tell who made it and how to cite it?
3. **Runnable** — is there any record of what it takes to run?
4. **Findable** — would anyone outside the lab ever discover it?

The goal is the **single highest-value next step**, not fixing everything.

**Try it now:** open your fork of [software-demo](#) and spend 2 minutes answering all four for yourself. We'll close every gap you find over the next episodes.

# Key points

---

- Software is a research output the scholarly system has mostly failed to catch
- A GitHub URL is not a citation — DOIs are permanent and version-specific, URLs rot
- Undeposited software has a bus-factor problem; forges do shut down
- FAIR4RS (Findable, Accessible, Interoperable, Reusable) is the framework
- Public repositories increase findability, reuse potential, and citation credit
- You'll build up your own fork step by step, episode by episode